JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.logging

Class FileHandler

java.lang.Object
  java.util.logging.Handler
    java.util.logging.StreamHandler
      java.util.logging.FileHandler

```
public class FileHandler
extends StreamHandler
```

Simple file logging `Handler`.

The `FileHandler` can either write to a specified file, or it can write to a rotating set of files.

For a rotating set of files, as each file reaches a given size limit, it is closed, rotated out, and a new file opened. Successively older files are named by adding "0", "1", "2", etc. into the base filename.

By default buffering is enabled in the IO libraries but each log record is flushed out when it is complete.

By default the `XMLFormatter` class is used for formatting.

Configuration: By default each `FileHandler` is initialized using the following `LogManager` configuration properties where `<handler-name>` refers to the fully-qualified class name of the handler. If properties are not defined (or have invalid values) then the specified default values are used.

- <handler-name>.level specifies the default level for the `Handler` (defaults to `Level.ALL`).
- <handler-name>.filter specifies the name of a `Filter` class to use (defaults to no `Filter`).
- <handler-name>.formatter specifies the name of a `Formatter` class to use (defaults to `java.util.logging.XMLFormatter`)

- <handler-name>.encoding the name of the character set encoding to use (defaults to the default platform encoding).
- <handler-name>.limit specifies an approximate maximum amount to write (in bytes) to any one file. If this is zero, then there is no limit. (Defaults to no limit).
- <handler-name>.count specifies how many output files to cycle through (defaults to 1).
- <handler-name>.pattern specifies a pattern for generating the output file name. See below for details. (Defaults to "%h/java%u.log").
- <handler-name>.append specifies whether the FileHandler should append onto any existing files (defaults to false).

For example, the properties for `FileHandler` would be:

- java.util.logging.FileHandler.level=INFO
- java.util.logging.FileHandler.formatter=java.util.logging.SimpleFormatter

For a custom handler, e.g. com.foo.MyHandler, the properties would be:

- com.foo.MyHandler.level=INFO
- com.foo.MyHandler.formatter=java.util.logging.SimpleFormatter

A pattern consists of a string that includes the following special components that will be replaced at runtime:

- "/" the local pathname separator
- "%t" the system temporary directory
- "%h" the value of the "user.home" system property
- "%g" the generation number to distinguish rotated logs
- "%u" a unique number to resolve conflicts
- "%%" translates to a single percent sign "%"

If no "%g" field has been specified and the file count is greater than one, then the generation number will be added to the end of the generated filename, after a dot.

Thus for example a pattern of "%t/java%g.log" with a count of 2 would typically cause log files to be written on Solaris to /var/tmp/java0.log and /var/tmp/java1.log whereas on Windows 95 they would be typically written to C:\TEMP\java0.log and C:\TEMP\java1.log

Generation numbers follow the sequence 0, 1, 2, etc.

Normally the "%u" unique field is set to 0. However, if the `FileHandler` tries to open the filename and finds the file is currently in use by another process it will increment the unique number field and try again. This will be repeated until `FileHandler` finds a file name that is not currently in use. If there is a conflict and no "%u" field has been specified, it will be added at the end of the filename after a dot. (This will be after any automatically added generation number.)

Thus if three processes were all trying to log to fred%u.%g.txt then they might end up using fred0.0.txt, fred1.0.txt, fred2.0.txt as the first file in their rotating sequences.

Note that the use of unique ids to avoid conflicts is only guaranteed to work reliably when using a local disk file system.

Since:

1.4

Constructor Summary

Constructors

Constructor	Description
`FileHandler()`	Construct a default `FileHandler`.
`FileHandler(String pattern)`	Initialize a `FileHandler` to write to the given filename.
`FileHandler(String pattern, boolean append)`	Initialize a `FileHandler` to write to the given filename, with optional append.
`FileHandler(String pattern, int limit, int count)`	Initialize a `FileHandler` to write to a set of files.
`FileHandler(String pattern, int limit, int count, boolean append)`	Initialize a `FileHandler` to write to a set of files with optional append.

Method Summary

Concrete Methods

Modifier and Type	Method	Description

void	**close**()	Close all the files.
void	**publish**(**LogRecord** record)	Format and publish a LogRecord.

Methods inherited from class java.util.logging.StreamHandler

flush, isLoggable, setEncoding, setOutputStream

Methods inherited from class java.util.logging.Handler

getEncoding, getErrorManager, getFilter, getFormatter, getLevel, reportError, setErrorManager, setFilter, setFormatter, setLevel

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

FileHandler

```
public FileHandler()
            throws IOException,
                   SecurityException
```

Construct a default FileHandler. This will be configured entirely from LogManager properties (or their default values).

Throws:

IOException - if there are IO problems opening the files.

SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control")).

NullPointerException - if pattern property is an empty String.

FileHandler

```
public FileHandler(String pattern)
            throws IOException,
                   SecurityException
```

Initialize a `FileHandler` to write to the given filename.

The `FileHandler` is configured based on `LogManager` properties (or their default values) except that the given pattern argument is used as the filename pattern, the file limit is set to no limit, and the file count is set to one.

There is no limit on the amount of data that may be written, so use this with care.

Parameters:

`pattern - the name of the output file`

Throws:

`IOException - if there are IO problems opening the files.`

`SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").`

`IllegalArgumentException - if pattern is an empty string`

FileHandler

```
public FileHandler(String pattern,
                   boolean append)
            throws IOException,
                   SecurityException
```

Initialize a `FileHandler` to write to the given filename, with optional append.

The `FileHandler` is configured based on `LogManager` properties (or their default values) except that the given pattern argument is used as the filename pattern, the file limit is set to no limit, the file count is set to one, and the append mode is set to the given `append` argument.

There is no limit on the amount of data that may be written, so use this with care.

Parameters:

`pattern - the name of the output file`

`append - specifies append mode`

Throws:

`IOException - if there are IO problems opening the files.`

`SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").`

`IllegalArgumentException - if pattern is an empty string`

FileHandler

```
public FileHandler(String pattern,
                   int limit,
                   int count)
            throws IOException,
                   SecurityException
```

Initialize a `FileHandler` to write to a set of files. When (approximately) the given limit has been written to one file, another file will be opened. The output will cycle through a set of count files.

The `FileHandler` is configured based on `LogManager` properties (or their default values) except that the given pattern argument is used as the filename pattern, the file limit is set to the limit argument, and the file count is set to the given count argument.

The count must be at least 1.

Parameters:

`pattern` - the pattern for naming the output file

`limit` - the maximum number of bytes to write to any one file

`count` - the number of files to use

Throws:

`IOException` - if there are IO problems opening the files.

`SecurityException` - if a security manager exists and if the caller does not have `LoggingPermission("control")`.

`IllegalArgumentException` - if `limit < 0`, or `count < 1`.

`IllegalArgumentException` - if pattern is an empty string

FileHandler

```
public FileHandler(String pattern,
                   int limit,
                   int count,
                   boolean append)
            throws IOException,
                   SecurityException
```

Initialize a `FileHandler` to write to a set of files with optional append. When (approximately) the given limit has been written to one file, another file will be opened. The output will cycle through a set of count files.

The `FileHandler` is configured based on `LogManager` properties (or their default values) except that the given pattern argument is used as the

filename pattern, the file limit is set to the limit argument, and the file count is set to the given count argument, and the append mode is set to the given append argument.

The count must be at least 1.

Parameters:

`pattern - the pattern for naming the output file`

`limit - the maximum number of bytes to write to any one file`

`count - the number of files to use`

`append - specifies append mode`

Throws:

`IOException - if there are IO problems opening the files.`

`SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").`

`IllegalArgumentException - if limit < 0, or count < 1.`

`IllegalArgumentException - if pattern is an empty string`

Method Detail

publish

```
public void publish(LogRecord record)
```

Format and publish a `LogRecord`.

Overrides:

`publish in class StreamHandler`

Parameters:

`record - description of the log event. A null record is silently ignored and is not published`

close

```
public void close()
           throws SecurityException
```

Close all the files.

Overrides:

`close in class StreamHandler`

Throws:

`SecurityException` - if a security manager exists and if the caller does not have LoggingPermission("control").

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.